

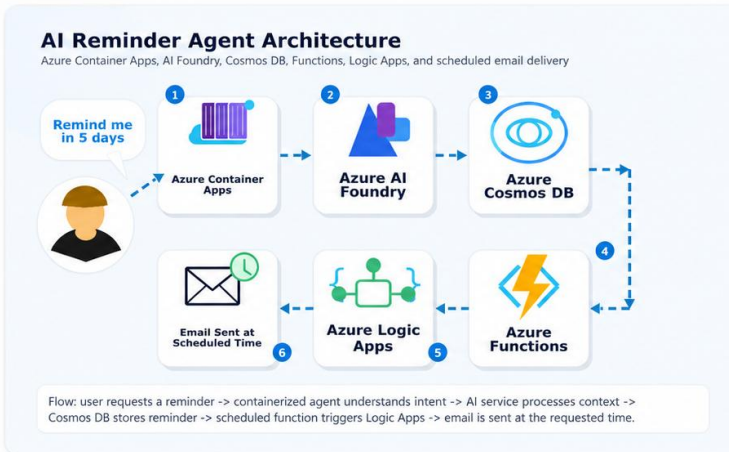
Email Reminder Agent

Employer-Facing Project Case Study

AI-enabled reminder scheduling with Azure Container Apps, Azure AI Foundry, Cosmos DB, Azure Functions, Logic Apps, and email delivery automation

01

Email Reminder Agent – Project Overview

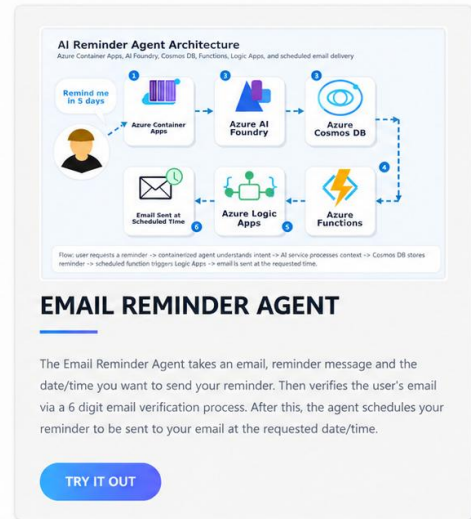


Project Summary

An intelligent agent that captures reminder requests, validates email addresses, and automatically sends reminders at the requested date and time.

- Accepts reminder details and requested send time
- Verifies the user's email before scheduling
- Sends the reminder automatically at the scheduled time

🔒 Sensitive details blurred for documentation.



Project Type	AI-powered cloud automation / full-stack portfolio project
Business Value	Automates reminder scheduling, verification, storage, and timed email delivery
Core Azure Services	Container Apps, AI Foundry, Cosmos DB, Functions, Logic Apps, Container Registry
Documentation Note	Screenshots have sensitive identifiers, URLs, emails, and subscription details blurred

Prepared as a professional portfolio case study. The live endpoint should be replaced on the portfolio with a "View Case Study" or "View Documentation" button.

Table of Contents

- 1. Executive Summary
- 2. Problem Statement and Business Use Case
- 3. Solution Goals
- 4. Architecture Overview
- 5. Azure Resource Design
- 6. Containerized Backend and AI Layer
- 7. Data Storage and Scheduled Processing
- 8. Email Delivery Workflow
- 9. Verification and End-to-End Result
- 10. Security, Privacy, and Documentation Hygiene
- 11. Monitoring and Operational Validation
- 12. Skills Demonstrated to Employers
- 13. Future Improvements
- 14. Conclusion

1. Executive Summary

The Email Reminder Agent is an AI-enabled cloud application that allows a user to submit an email reminder request, verify the recipient email address, persist the reminder, and automatically send the reminder at the requested time.

The project demonstrates practical Azure integration across application hosting, AI service orchestration, database storage, serverless scheduled processing, workflow automation, and email delivery. It is designed as a portfolio-ready case study that communicates both technical implementation and business value.

From an employer perspective, the project shows that the developer can move beyond isolated tutorials and assemble multiple cloud services into a working end-to-end solution with verification, persistence, automation, and evidence of successful execution.

Employer-Relevant Highlights

Area	What the Project Shows	Why It Matters
Cloud Deployment	Backend hosted as a container app	Shows practical Azure deployment and managed runtime knowledge
AI Integration	Azure AI Foundry included in the reminder workflow	Shows awareness of modern AI-enabled application architecture
Data Persistence	Reminder records stored in Cosmos DB	Shows database-backed workflow design instead of temporary in-memory logic
Automation	Timer-triggered Function and Logic App delivery	Shows serverless and workflow automation skills
Validation	Email verification before scheduling	Shows attention to reliability and user trust

2. Problem Statement and Business Use Case

Many reminder systems depend on a user keeping a browser session open, manually checking a task list, or relying on a single application to stay available. This project solves a practical problem: a user should be able to request a reminder once and trust the system to deliver it later without further manual action.

The business use case is simple but realistic. A user provides an email address, reminder message, and desired delivery time. The system verifies the email address, schedules the reminder, stores it safely, and sends the reminder when it becomes due.

This kind of solution can be adapted for appointment reminders, follow-up notifications, customer support nudges, HR reminders, student alerts, billing reminders, and workflow-driven business notifications.

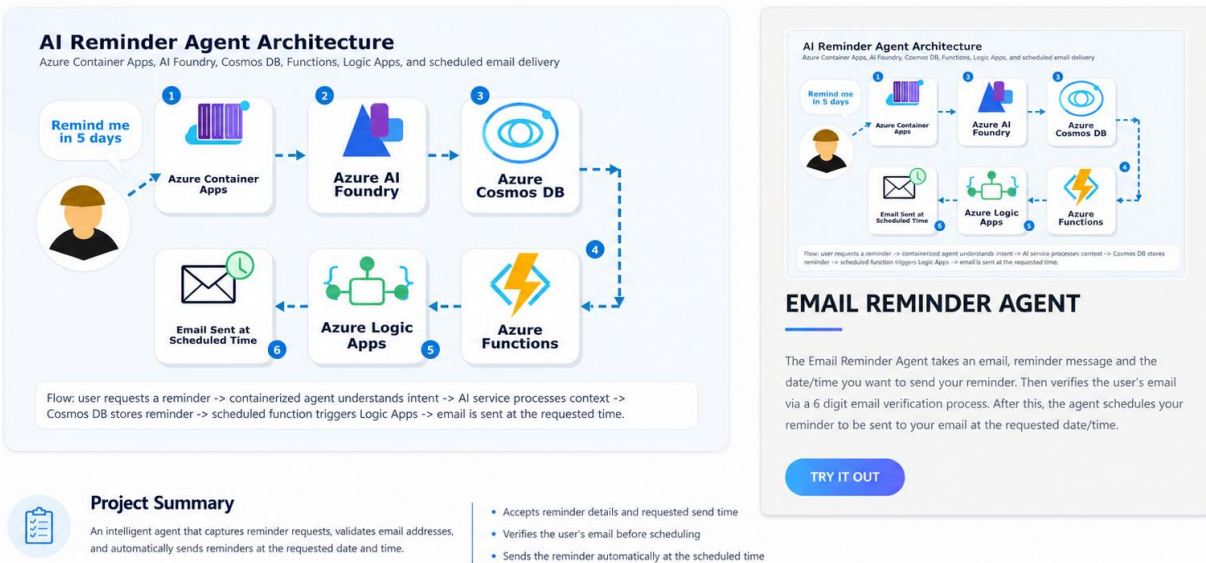
3. Solution Goals

- Create a simple reminder request experience that is easy to demonstrate in a portfolio.
- Verify the email address before storing and delivering a reminder.
- Persist reminder records so scheduled work survives beyond the original request.
- Use cloud-native scheduled processing to identify reminders that are due.
- Use Logic Apps to handle email delivery in a visual, maintainable workflow.
- Document the system in a way that proves the project works without exposing live endpoints or sensitive cloud details.

4. Architecture Overview

01

Email Reminder Agent – Project Overview



Sensitive details blurred for documentation.

Figure 1. High-level architecture and portfolio overview for the Email Reminder Agent.

The architecture begins with the user submitting a reminder request. The request is handled by a containerized backend service hosted in Azure Container Apps. The AI layer, represented by Azure AI Foundry, supports the intelligent processing context for the reminder workflow.

After the request is processed and verified, the reminder information is stored in Azure Cosmos DB. A scheduled Azure Function later checks for reminders that have reached their requested send time. When a reminder is due, the Function App triggers the Logic App, which sends the final reminder email.

This design intentionally separates responsibilities: the backend receives and validates requests, the database stores reminder records, the Function App performs scheduled checks, and the Logic App handles email delivery.

Architecture Flow

1. User submits reminder details and requested delivery time.
2. Containerized backend receives the request and coordinates the workflow.
3. Email verification is sent before scheduling the reminder.
4. Verified reminder records are saved in Cosmos DB.
5. Timer-triggered Azure Function checks for due reminders.
6. Logic App sends the final email when triggered.
7. The user receives the reminder at the scheduled time.

5. Azure Resource Design

2

Solution Architecture & Azure Resources

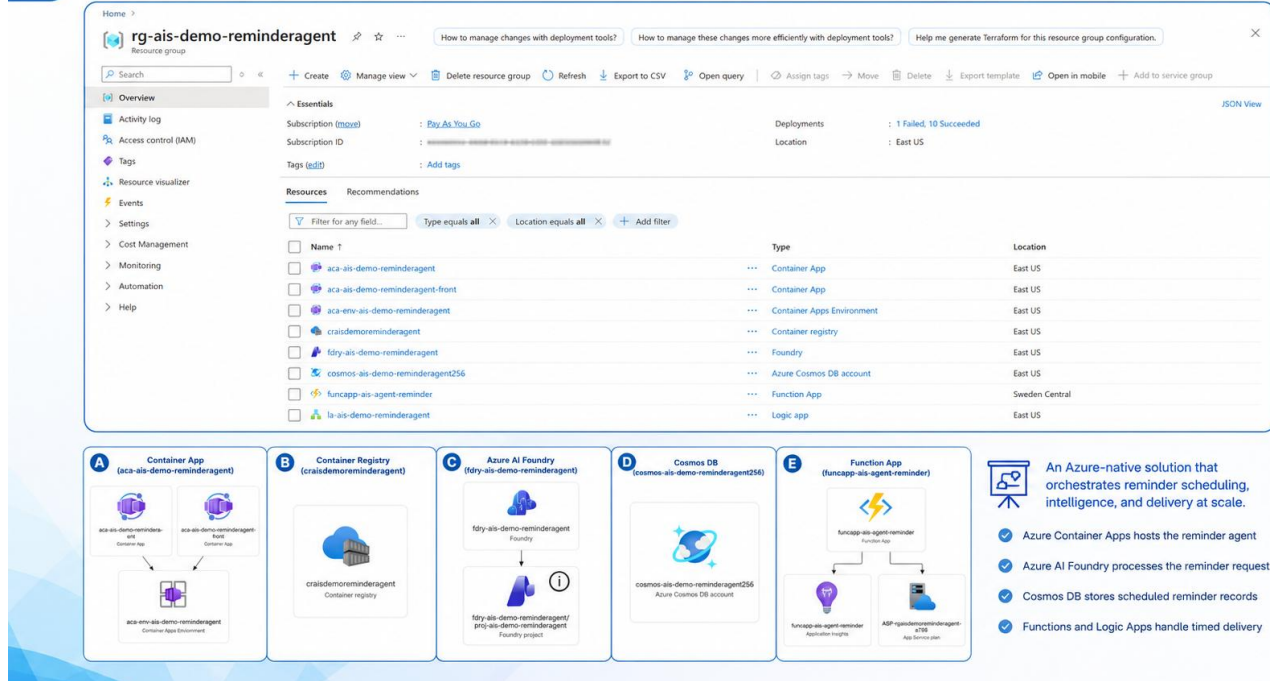


Figure 2. Azure resource group and major services used by the solution.

The Azure resource group provides a consolidated view of the services required for the solution. This includes the backend container application, frontend container application, container environment, container registry, AI Foundry resource, Cosmos DB account, Function App, Application Insights, App Service Plan, and Logic App.

For an employer review, this screenshot is important because it proves the project was not only coded locally. It was deployed and connected to real Azure services as a working cloud solution.

The resource visualizer panels also communicate how the services relate to each other. This helps reviewers quickly understand the cloud footprint without needing access to the live Azure subscription.

Service Responsibility Matrix

Service	Responsibility	Portfolio Value
Azure Container Apps	Hosts the backend reminder agent API	Demonstrates containerized cloud deployment
Azure Container Registry	Stores container images used for deployment	Demonstrates image management and deployment readiness
Azure AI Foundry	Provides AI project/service layer	Demonstrates AI service integration
Azure Cosmos DB	Stores pending and scheduled reminder records	Demonstrates cloud database design
Azure Functions	Runs timer-triggered scheduled processing	Demonstrates serverless automation
Azure Logic Apps	Sends the final reminder email	Demonstrates workflow orchestration
Application Insights	Supports monitoring and visibility	Demonstrates operational awareness

6. Containerized Backend and AI Layer

03 Containerized Deployment & AI Layer

The backend API is containerized and deployed on Azure Container Apps for a scalable, managed runtime. Container images are securely stored in Azure Container Registry, and Azure AI Foundry powers the AI capabilities used by the reminder workflow.

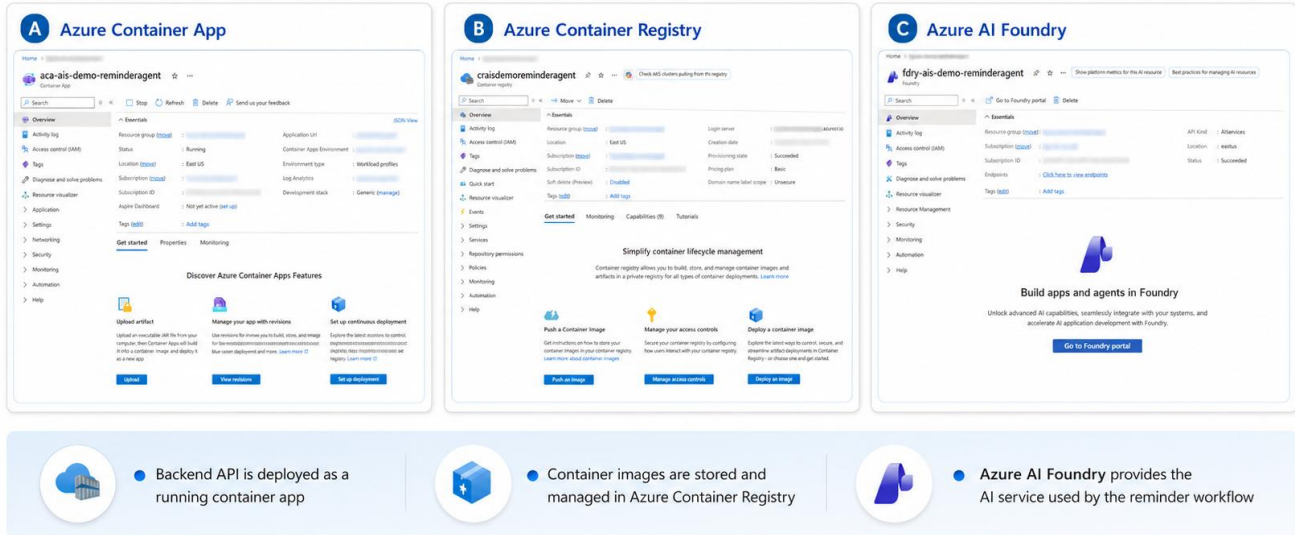


Figure 3. Azure Container Apps, Container Registry, and Azure AI Foundry screenshots.

The backend service is packaged as a container and deployed to Azure Container Apps. This is a strong implementation choice for a portfolio project because it shows familiarity with containerized deployment, cloud runtime configuration, and scalable hosting.

Azure Container Registry is used to store the application image. This creates a professional deployment flow where the image is built, pushed to a registry, and then consumed by the runtime environment.

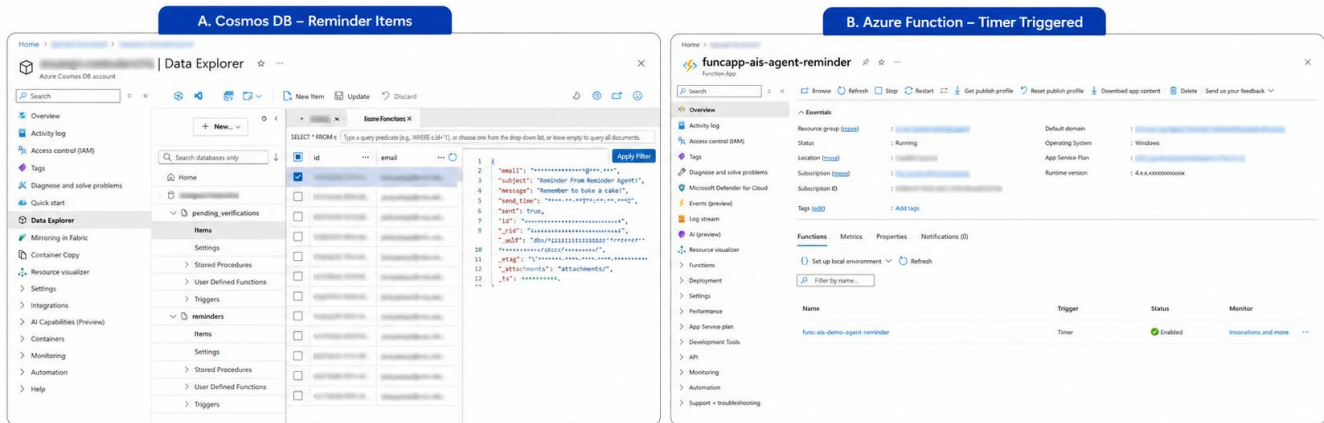
Azure AI Foundry represents the AI layer. In the architecture, it shows that the application is designed to use modern AI capabilities instead of being a basic static workflow. This is especially relevant for .NET, Azure, and AI automation roles.

What this proves to an employer

- The project can be packaged and deployed outside the local development machine.
- The developer understands the difference between source code, container images, registries, and runtime hosting.
- The project connects traditional backend engineering with newer AI-enabled application architecture.

7. Data Storage and Scheduled Processing

4 Data Storage & Scheduled Processing



Reminder records are securely stored in Cosmos DB. A timer-triggered Azure Function runs on a schedule to find reminders that are due and trigger delivery.

- ✓ Reminder details are saved in Cosmos DB
- ✓ A timer-triggered Azure Function checks for reminders that are due
- ✓ This enables delayed, automated reminder delivery

Figure 4. Cosmos DB reminder records and timer-triggered Azure Function.

Cosmos DB stores reminder records after verification. This is important because scheduled reminders must persist beyond the original user request. Without persistent storage, a reminder system would be unreliable after restarts or redeployments.

The Data Explorer screenshot shows reminder-related containers and stored item records. Sensitive values such as emails, identifiers, and internal document metadata have been blurred, but the structure still communicates that reminders are stored as database records.

The timer-triggered Azure Function is responsible for checking reminders on a schedule. This separates scheduled processing from request handling and makes the application more reliable and maintainable.

Data and Scheduling Responsibilities

- Cosmos DB stores pending verifications and reminder items.
- Each reminder includes the recipient, subject, message, scheduled send time, and delivery status.
- The Function App checks for due reminders based on time.
- When a reminder is due, the Function App triggers the delivery workflow.

8. Email Delivery Workflow

5 Email Delivery Workflow

This Logic App automates sending reminder emails. It is triggered by an HTTP request and uses the Outlook mail connector to deliver the email.

- ✓ The Logic App receives an HTTP request
- ✓ It sends the reminder email through the configured mail connector
- ✓ Run history confirms successful workflow executions

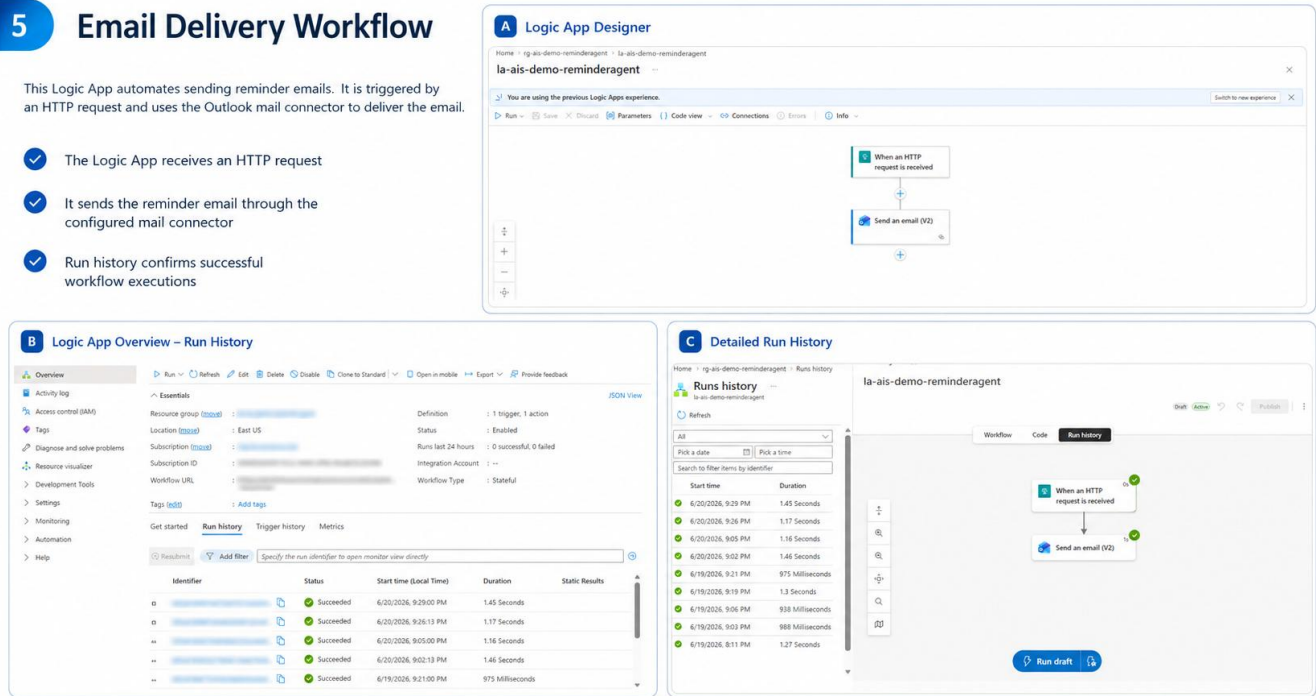


Figure 5. Logic App designer and successful run history.

The Logic App handles the final email delivery step. It receives an HTTP request and then uses an email connector to send the reminder message. The workflow is intentionally simple and visual, making it easy to maintain and explain.

The run history confirms that the workflow executed successfully. This is valuable evidence for a portfolio because it shows that the system was tested and actually delivered output, rather than only being designed on paper.

The use of Logic Apps also demonstrates practical workflow automation skills, which are highly relevant for Power Platform, Azure integration, and business automation roles.

9. Verification and End-to-End Result

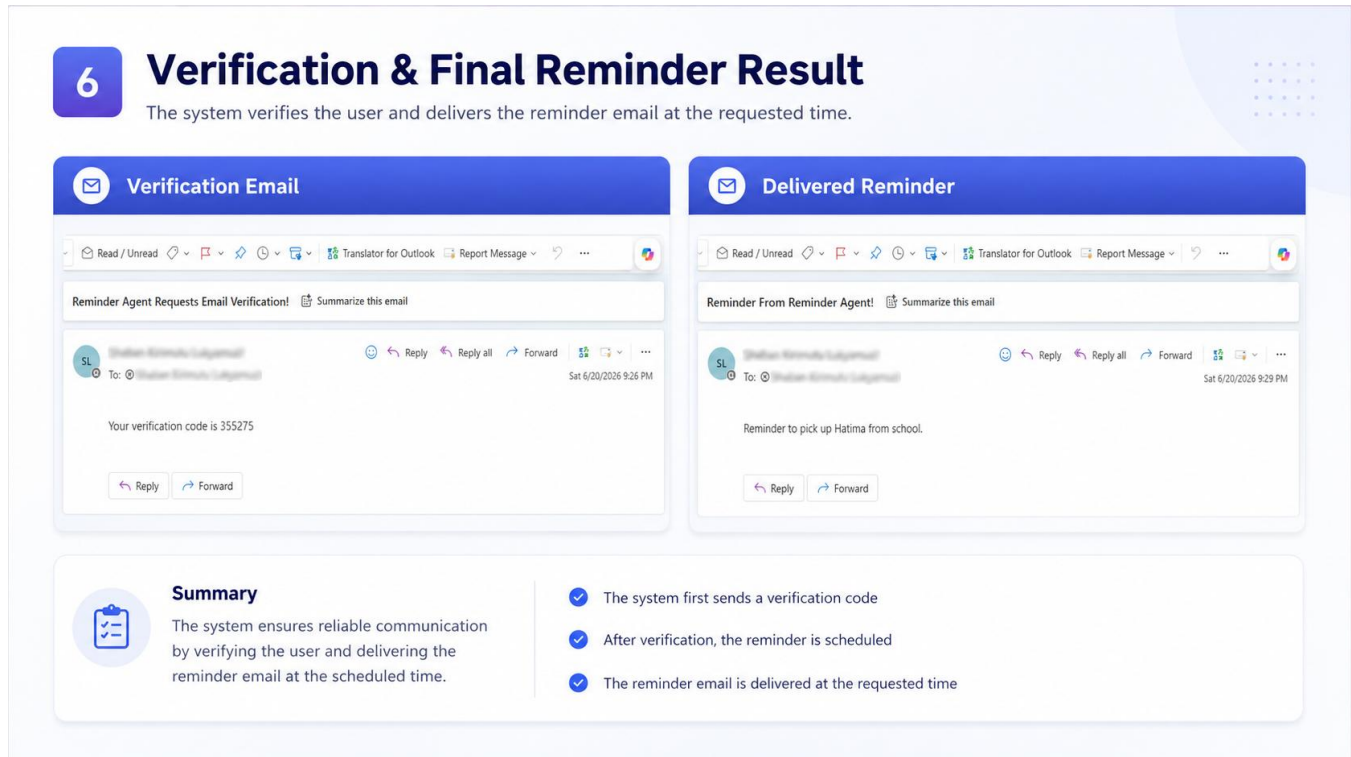


Figure 6. Verification email and final reminder email delivered to the user.

The first email demonstrates the verification step. Before a reminder is accepted as scheduled, the user receives a six-digit verification code. This improves reliability by confirming that the email address belongs to the user requesting the reminder.

The second email demonstrates the final result: the reminder is delivered at the scheduled time. Together, these screenshots show the complete workflow from verification to final delivery.

This is one of the strongest pieces of evidence in the case study because it confirms that the cloud services are not only configured but are working together to produce the intended business outcome.

10. Security, Privacy, and Documentation Hygiene

Because this project is being prepared for a public portfolio, sensitive information should not be exposed in screenshots or live links. The live application URL, subscription ID, workflow URL, email addresses, document IDs, endpoints, and account details should be blurred or removed before publishing.

The portfolio should avoid direct “Try It Out” links for this project if the live services are not intended for public use. A safer approach is to replace the button with “View Case Study” or “View Documentation.”

This case study intentionally uses blurred screenshots so the project can be reviewed by employers without exposing the Azure subscription or user data.

Recommended items to hide in public documentation

- Azure subscription ID, tenant ID, client IDs, and resource endpoints.
- Container App application URLs and Logic App workflow callback URLs.
- Function keys, connection strings, API keys, and environment variable values.

- Real email addresses, user names, reminder messages, and verification codes.
- Cosmos DB document IDs, partition keys, and internal metadata where unnecessary.

11. Monitoring and Operational Validation

The project includes operational evidence through Logic App run history and Function App status. These views help validate that the automation is enabled and that workflows have successfully completed.

For a production version, Application Insights and Azure Monitor could be used more deeply to track request volume, failed deliveries, function execution duration, exception details, and retry behavior.

Including monitoring screenshots in the case study helps employers see that the project was tested and reviewed beyond the initial build phase.

12. Skills Demonstrated to Employers

Skill Area	Evidence in Project
Cloud Architecture	Designed a multi-service Azure solution with clear separation of responsibilities.
Containerization	Packaged and deployed the backend through Azure Container Apps and Container Registry.
AI Application Design	Integrated Azure AI Foundry as part of the intelligent processing layer.
Database Design	Stored verified reminder records in Cosmos DB for scheduled processing.
Serverless Automation	Used a timer-triggered Azure Function to process reminders at the right time.
Workflow Automation	Used Logic Apps to send reminder emails and track run history.
Security Awareness	Implemented email verification and blurred sensitive details for safe documentation.
Portfolio Communication	Converted technical implementation into an employer-friendly case study.

13. Future Improvements

- Add authentication so users can manage their own reminders securely.
- Add a dashboard for viewing, updating, cancelling, and rescheduling reminders.
- Add retry handling and dead-letter logic for failed email deliveries.
- Add Application Insights dashboards for request volume, failures, and delivery metrics.
- Add infrastructure-as-code using Bicep or Terraform for reproducible deployment.
- Add CI/CD with GitHub Actions to build the container image and deploy updates automatically.
- Add support for multiple reminder channels such as SMS, Teams, or push notifications.

14. Conclusion

The Email Reminder Agent demonstrates practical integration across backend development, Azure cloud services, AI tooling, serverless processing, workflow automation, database storage, and email communication.

The project is easy for non-technical reviewers to understand because the business use case is simple: submit a reminder, verify the email, store the request, and receive the reminder later. At the same time, the technical implementation shows advanced cloud integration skills.

The recommended portfolio presentation is to remove the live “Try It Out” link and replace it with a curated documentation or case study link. This protects sensitive resources while still allowing employers to evaluate the quality of the project.